FRR WorkShop

Donald Sharp, Principal Engineer NVIDIA



Agenda

- ASIC Offloading
- Netlink Batching
- Nexthop Group Expansion
- How To Get Involved
- Townhall

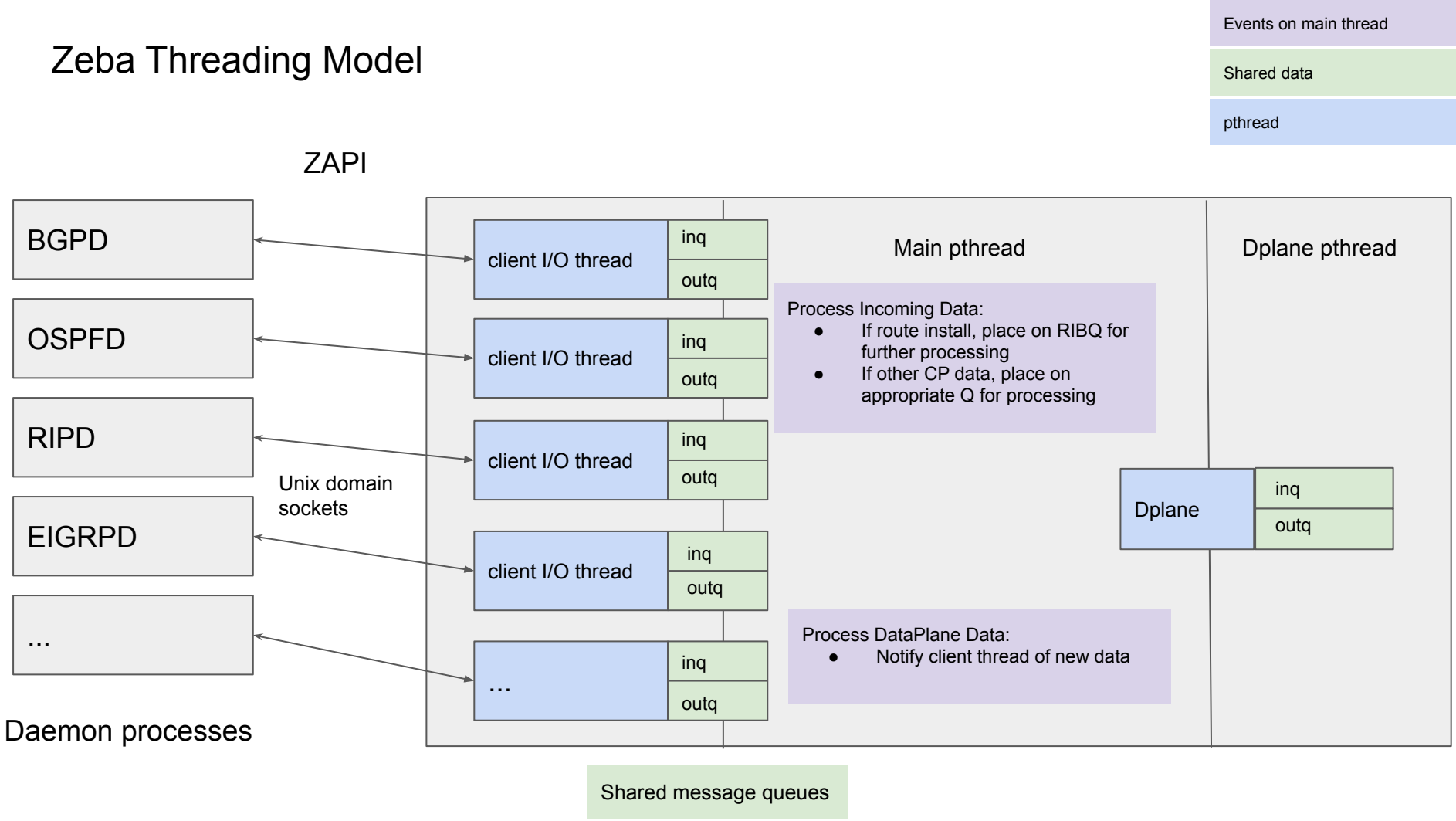
ASIC Offloading



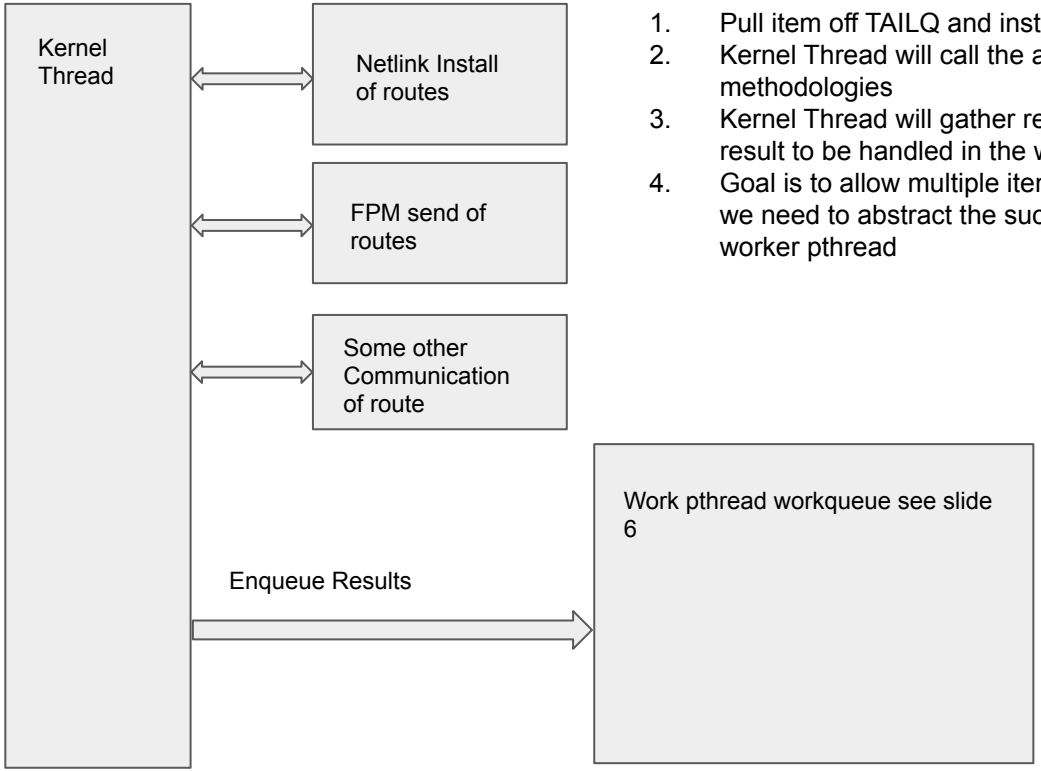
Motivation

- Kernel recently received ability to inform interested parties that routes are offloaded
  - RTM_F_OFFLOAD
  - RTM_F_TRAP
  - Commit ID’s
    - bb3c4ab93e44 ipv4: Add “offload” and “trap” indications to routes
    - 90b93f1b31f8 ipv6: Add “offload” and “trap” indications to routes
- FPM Always Implied a ASIC offload
- Need a way to notice!
  - Bits and Pieces of Code are already there, let’s connect the dots

Zeba Threading Model



DataPlane Thread



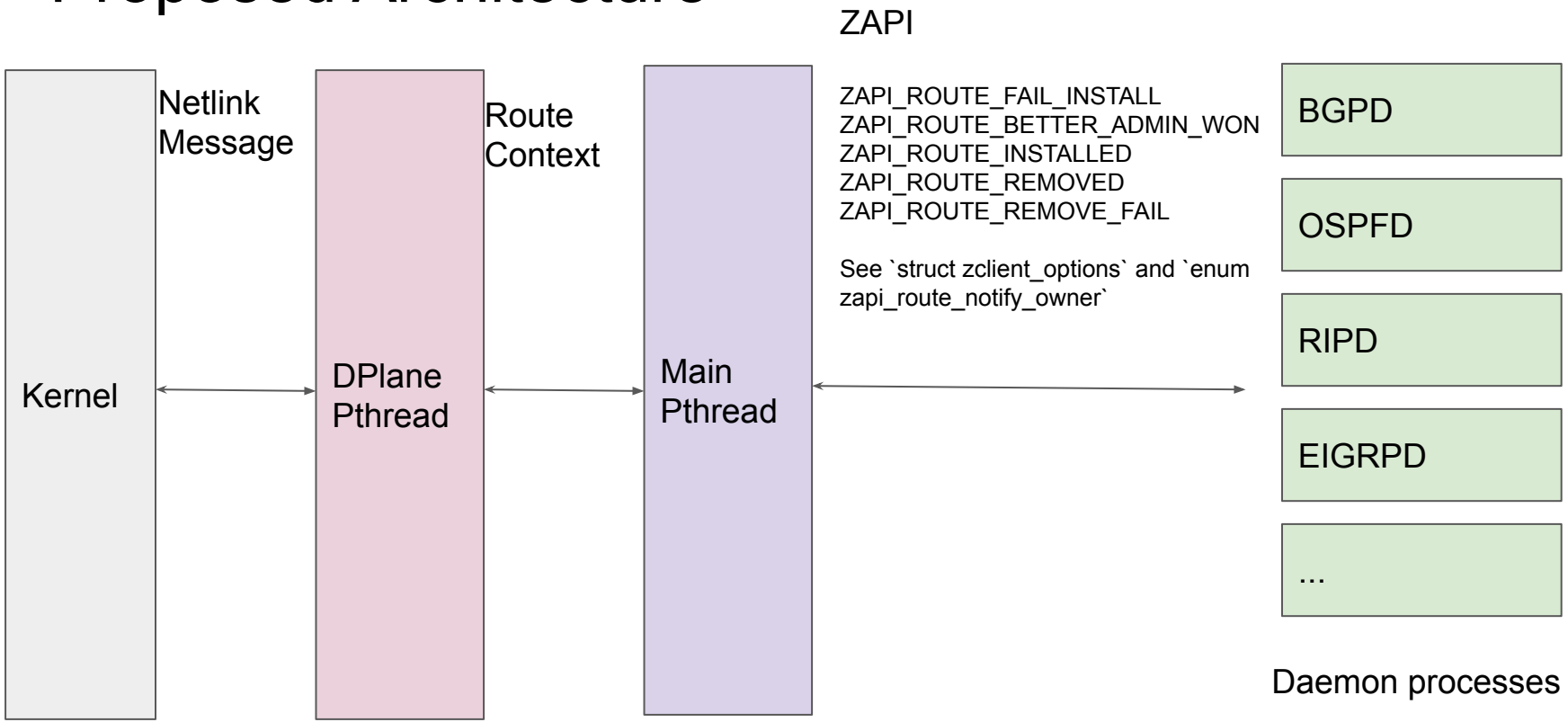
1. Pull item off TAILQ and install into kernel
2. Kernel Thread will call the appropriately hooked up communication methodologies
3. Kernel Thread will gather results from various methodologies and enqueue a result to be handled in the work pthread workqueue
4. Goal is to allow multiple items on the TAILQ to be handled at one time as such we need to abstract the success/failure and resulting setting of flags into the worker pthread

- Two Netlink Sockets
  - Command
  - Data has BPF Filter to limit reading our own Data

Proposed Architecture

- Watch netlink messages for new flags per route
- Add a ZEBRA_FLAG_FIB_FAIL when we receive a RTM_F_OFFLOAD flag clear from kernel
- FPM would need it’s own implementation to match into it
- Notify upper level owner protocols that something has gone wrong

Proposed Architecture



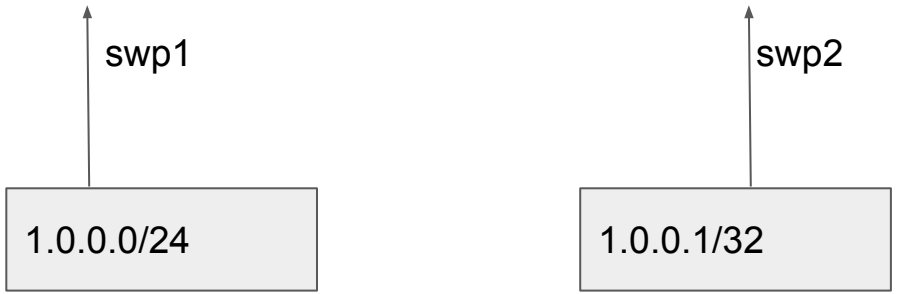
Proposed Architectures Continued

There is a Installation Issue with how Data is handled from The Kernel. This is the first time that the kernel will be setting flags on data we hand to it, so we need a way to know the state.

a) Turn off BPF filter
   - Note offload flag(s) from kernel and pass up to main pthread for Zebra Processing
b) Know that we are offloading (and for which set of interfaces) and just listen for offload failures
   - How do we know this? Not easy at the moment from an Upstream Perspective

What should BGP(Or any higher level protocol) do?

- Networking is Busted on Route installation Failure
  - Shutdown peering



References

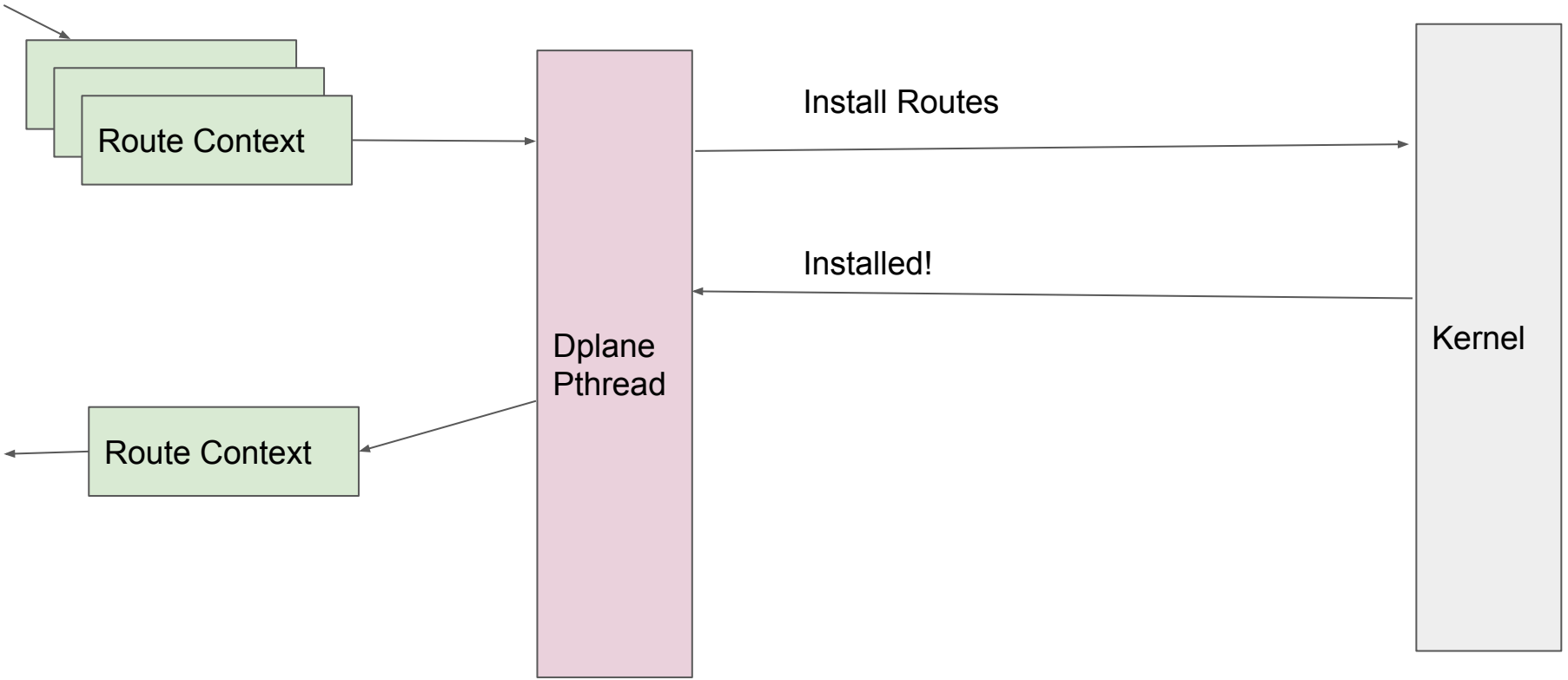
- Zebra Reference Presentation https://docs.google.com/presentation/d/1SeDS5b-Wgmp-2T_9povfHscP6Xpaihff_xsxWdTlKDE/edit?usp=sharing
- BGP Reference Presentation https://docs.google.com/presentation/d/107fjFyrjNwn9ogP-yuygD71Kx3CQtoqqDOzMKHBK2xM/edit#slide=id.p
- Available in FRR Slack

Netlink Batching

GSOC 2020 Programmed by Jakub Urbańczyk



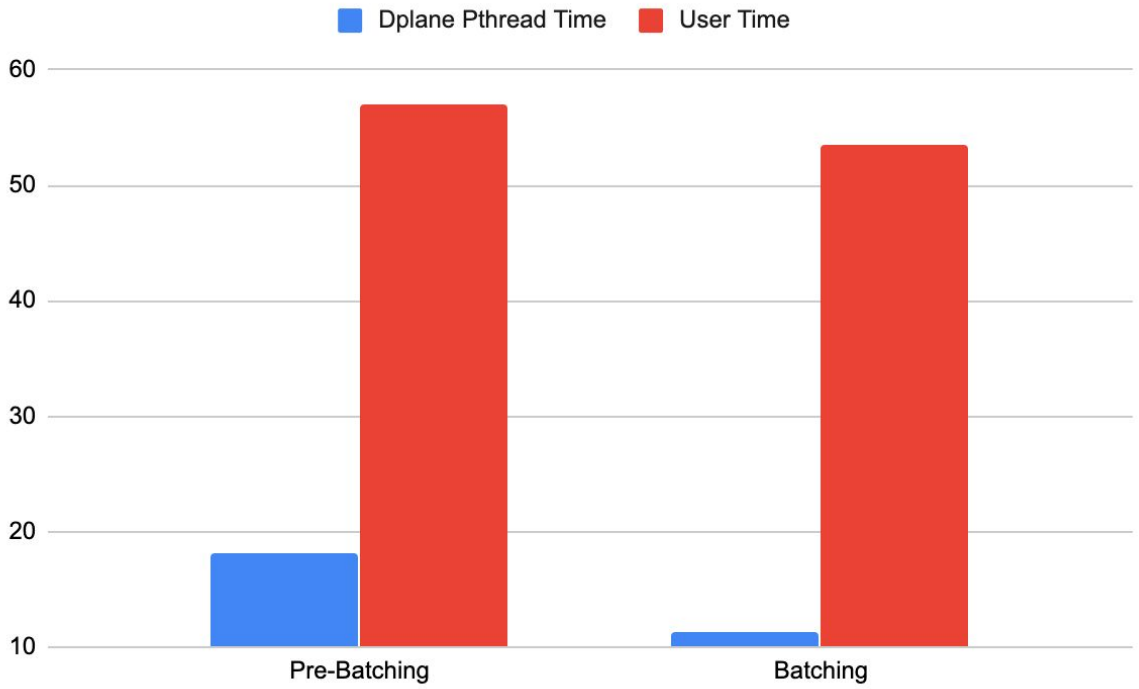
How It Works


Route Context
Install Routes
Installed!
Dplane
Pthread
Kernel
Route Context

What Have We Gained

- 1 Million Routes x 16 ECMP
- Time is in Seconds For Install and Delete

- Installation is ~25 seconds Wall time
- Deletion is ~28 seconds Wall time



References

https://github.com/xThaid/frr/wiki/Dataplane-batching

Nexthop Groups Continued

In Which an Upper Level Protocol Gets it



Motivation

- EVPN MH
  - See
    - https://github.com/FRRouting/frr/pull/6587 Initial Support for Type 1 Routes - IN Code Base
    - https://github.com/FRRouting/frr/pull/6883 Refactor - Merging Soon
    - https://github.com/FRRouting/frr/pull/6799 NHG Work - In Review
    - +1 to come
- BGP PIC
  - Future Work
- Greatly Speed up Route Installation from an Upper level Protocol
  - Need Convergence on new Forwarding Plane in a very small amount of time

FRR Nexthop Groups

- Zebra is done
  - See Previous FRR Workshop
- Each Daemon can manage its own space or let Zebra do so
- Zebra always manages individual Nexthops in it's own space

```
id 18 via 192.168.161.1 dev enp39s0 scope link proto zebra
id 19 via 192.168.161.1 dev enp39s0 scope link proto sharp
id 20 via 192.168.161.2 dev enp39s0 scope link proto sharp
id 21 via 192.168.161.3 dev enp39s0 scope link proto sharp
id 22 via 192.168.161.4 dev enp39s0 scope link proto sharp
id 23 via 192.168.161.5 dev enp39s0 scope link proto sharp
id 24 via 192.168.161.6 dev enp39s0 scope link proto sharp
id 25 via 192.168.161.7 dev enp39s0 scope link proto sharp
id 26 via 192.168.161.8 dev enp39s0 scope link proto sharp
id 27 via 192.168.161.9 dev enp39s0 scope link proto sharp
id 28 via 192.168.161.10 dev enp39s0 scope link proto sharp
id 29 via 192.168.161.11 dev enp39s0 scope link proto sharp
id 30 via 192.168.161.12 dev enp39s0 scope link proto sharp
id 31 via 192.168.161.13 dev enp39s0 scope link proto sharp
id 32 via 192.168.161.14 dev enp39s0 scope link proto sharp
id 33 via 192.168.161.15 dev enp39s0 scope link proto sharp
id 34 via 192.168.161.16 dev enp39s0 scope link proto sharp
id 36 via 192.168.161.11 dev enp39s0 scope link proto zebra
id 40 group 36/41 proto zebra
id 41 via 192.168.161.12 dev enp39s0 scope link proto zebra
id 185483868 group
19/20/21/22/23/24/25/26/27/28/29/30/31/32/33/34 proto sharp
```

Details

- Each Daemon is assigned it’s own NHG space
  - uint32_t space of Nexthop Groups
  - Upper 4 bits is for L2 Nexthop Groups (For EVPN MH)
  - Lower 28 bits are for Individual Protocols
    - Each Protocol gets ~8 million NHG’s
  - zclient_get_nhg_start(uint32_t proto)
    - Returns the starting spot for the proto (see lib/route_types.h)
    - Each daemon is expected to manage it’s own space
  - This API is optional

How Zebra/ZAPI Manages

- zclient_nhg_add(struct zclient *zclient, uint32_t id, size_t nhops, struct zapi_nexthop *znh);
  - Encapsulates Add and Replace semantics
- zclient_nhg_del(struct zclient *zclient, uint32_t id);
  - Removes the NHG id from the system
- Notification about events about your NHG's installed via
  - int (*nhg_notify_owner)(ZAPI_CALLBACK_ARGS);
    - ZAPI_NHG_FAIL_INSTALL
    - ZAPI_NHG_INSTALLED
    - ZAPI_NHG_REMOVED
    - ZAPI_NHG_REMOVE_FAIL
- Stores passed down NHG's in the NHG hash automatically

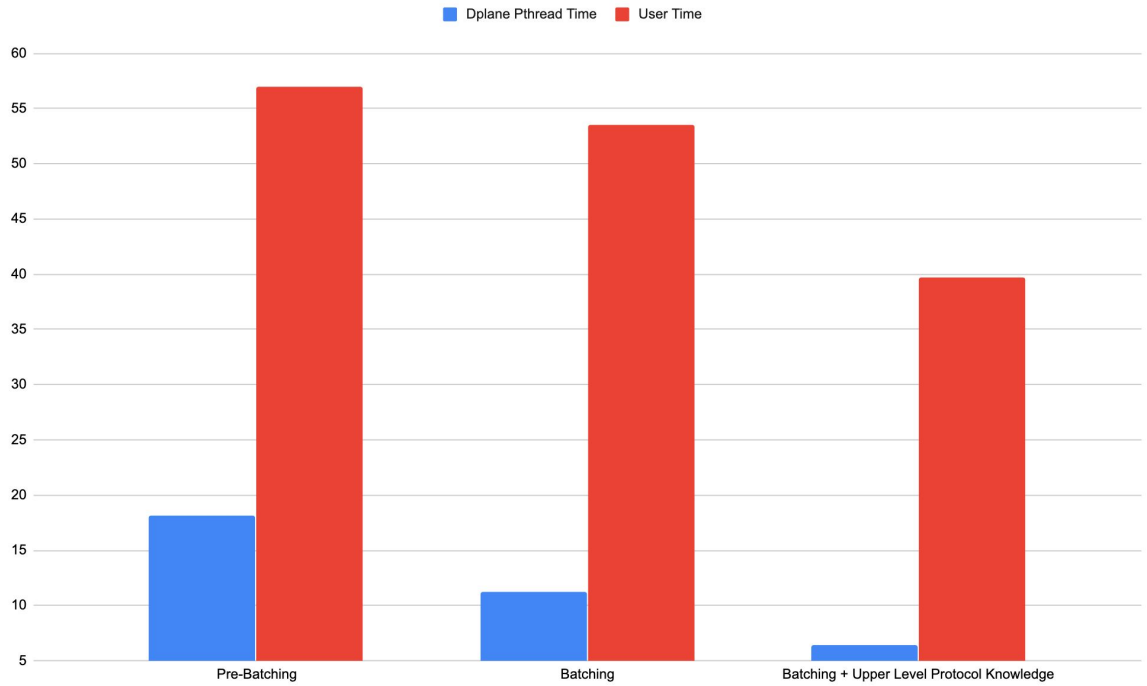
ZAPI NHG Benefits

- Passing a uint32_t(4 bytes)
- Minimum nexthop encoding per route is 7 bytes for 1xecmp
- Maximum nexthop encoding per route can be ~80 bytes or more for 1xecmp!
- Really adds up if you have large ecmp

What Have We Gained

- 1 Million Routes x 16 ECMP
- Time is in Seconds For Install and Delete
- Installation and Deletion is now Functionally Equivalent to 1xECMP

- Installation is ~10 seconds wall time
- Deletion is ~30 seconds wall time



How To Get Involved

- https://frrouting.org
  - Click on Slack link to join Slack
- https://lists.frrouting.org/listinfo
- Weekly Technical Meeting
  - Send me an email `sharpd AT nvidia dot com` asking to be included

